



SPRING FRAMEWORK

Spring Framework is a **powerful, lightweight, open-source framework** for building **Java-based enterprise applications**. It was developed by **Rod Johnson** and released in **2003** to address the complexities of enterprise application development using Java EE (J2EE).

Spring is not just one framework but a **comprehensive programming and configuration model** for modern Java-based enterprise apps — on any kind of deployment platform.

Core Features of Spring Framework

1. Lightweight and Modular

- Only the modules you need can be used; no need to load everything.
- Very lightweight in terms of memory and processing.

2. Inversion of Control (IoC)

- Objects are given their dependencies at runtime by the container.
- Promotes **loose coupling** and **better testability**.

3. Dependency Injection (DI)

- Objects are injected rather than creating dependencies manually.
- Makes code easier to test and maintain.

4. Aspect-Oriented Programming (AOP)

- Separates cross-cutting concerns (e.g., logging, security).
- Allows you to focus on business logic.

5. Transaction Management

- Abstract layer over different transaction management APIs (JTA, JDBC, Hibernate).
- Makes transaction handling consistent.

6. Spring MVC

- A powerful web framework for developing web applications.
- Follows Model-View-Controller architecture.

7. Integration with ORM

- Easily integrates with Hibernate, JPA, JDBC, etc.

8. Spring Security

- Powerful authentication and authorization system.

9. Spring Boot

- Built on Spring. Helps create standalone applications with **zero configuration**.
- Embedded server (Tomcat/Jetty), auto-configuration.

How Spring Solves These Problems

Problem	Spring Solution
Tightly Coupled Code	IoC & DI – Loose coupling
Complex Config	Annotation-based & Java Config
Heavy Containers	Lightweight POJOs, Spring Boot with embedded server
Difficult Testing	Easy unit testing with MockBeans
Boilerplate Code	Abstractions like JdbcTemplate, TransactionTemplate

Inversion of Control (IoC) in Spring

What is Inversion of Control (IoC)?

Inversion of Control (IoC) is a **design principle** where the **control of creating and managing objects is transferred from the application code to a container or framework**.

In traditional programming, the **developer creates and manages objects** using new keyword. In IoC, the **Spring Container is responsible** for:

- Creating the objects (beans)
- Configuring them
- Wiring their dependencies
- Managing their entire lifecycle

 **“Inversion” means: instead of you controlling the flow, Spring now controls the object creation and injection.**

Why IoC is Important?

Traditional Java	With IoC (Spring)
Developer writes code to create and manage objects manually	Spring Container handles creation and wiring
Code is tightly coupled	Code is loosely coupled
Difficult to test (due to hardcoded dependencies)	Easy to test using mock beans
Changing implementations is hard	Easily swappable via config or annotations

1. Bean

- A **bean** is an object that is **managed by the Spring IoC container**.

- Beans are instantiated, assembled, and managed by Spring.

Definition: “A bean is simply a POJO that has been registered with the Spring container.”

2. POJO (Plain Old Java Object)

- Simple Java class **without any special constraints**.
- No need to extend or implement any framework-specific interfaces.
- Example:

```
public class Student {
    private String name;

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }
}
```

Spring Bean Class

- Any POJO that is **annotated or defined** in Spring configuration to be managed as a bean.
- Example:

@Component

```
public class Student {
    // now it's a Spring-managed bean
}
```

Spring Bean Lifecycle

What is a Bean Lifecycle?

In Spring, when an object is defined as a **Bean**, its **creation, initialization, and destruction** are handled by the **Spring IoC container**. This entire process is known as the **Bean Lifecycle**.

Spring provides **hooks at various points** of the lifecycle to allow **custom logic** to be executed, such as logging, validation, initialization, and cleanup.

Phases of Spring Bean Lifecycle

Let's break it down into **10 important steps** for clarity:

1. Instantiation

- Spring **creates an object** (bean) using the Java constructor.
- Either default constructor or parameterized (if configured).

```
public class Student {
```

```
public Student() {  
    System.out.println("Step 1: Student bean instantiated");  
}  
}
```

✿ 2. Populate Properties (Dependency Injection)

- Spring **injects dependencies** using `@Autowired`, `@Value`, or XML `<property>`.

```
@Autowired  
private Subject subject;
```

✿ 3. BeanNameAware Interface

- If bean implements `BeanNameAware`, Spring passes the **ID** (name) of the bean.

```
public class MyBean implements BeanNameAware {  
    public void setBeanName(String name) {  
        System.out.println("Step 3: Bean name is: " + name);  
    }  
}
```

✿ 4. BeanFactoryAware Interface

- Lets you access the **BeanFactory** (basic IoC container).
-

✿ 5. ApplicationContextAware Interface

- Provides access to the **ApplicationContext** (advanced IoC container).

```
public class MyBean implements ApplicationContextAware {  
    public void setApplicationContext(ApplicationContext context) {  
        System.out.println("ApplicationContext received");  
    }  
}
```

✿ 6. Pre-Initialization Bean Post Processors

- You can define a **custom logic** to be run before the initialization (e.g., logging, validation).

```
public Object postProcessBeforeInitialization(Object bean, String beanName)
```

✿ 7. InitializingBean Interface / init-method

- You can add **initialization logic** (like loading resources, database connections).

```
public class MyBean implements InitializingBean {  
    public void afterPropertiesSet() {  
        System.out.println("Step 7: Initialization logic here");  
    }  
}
```

OR

```
<bean id="myBean" class="MyBean" init-method="init"/>
```

✿ 8. Custom Init Method

- Declared using `@PostConstruct` or `init-method` in XML.

`@PostConstruct`

```
public void init() {  
    System.out.println("Custom init method called");  
}
```

✿ 9. Destruction / Cleanup

- When container is shut down, Spring will **destroy the bean**.

You can:

- Implement `DisposableBean`
- Use `@PreDestroy` annotation
- Use `destroy-method` in XML

`@PreDestroy`

```
public void cleanup() {  
    System.out.println("Step 9: Bean cleanup before destruction");  
}
```

❧ 10. Bean Destroyed

Summary Diagram of Bean Lifecycle (for interviews):

Instantiation →

Dependency Injection →

Aware Interfaces (BeanNameAware, ApplicationContextAware) →

BeanPostProcessor (pre) →

Initialization (init-method or InitializingBean) →

BeanPostProcessor (post) →

Bean Ready →

Shutdown →

Destruction (@PreDestroy or DisposableBean)

BeanFactory

◆ Definition:

- **Basic container**, defined in `org.springframework.beans.factory.BeanFactory`
- **Lazily loads** beans — creates beans **only when requested**
- Useful for **lightweight applications** or mobile apps

When to Use:

- Simple applications with few beans
- Memory-constrained devices
- Manual control needed over instantiation

Advantages:

- Faster startup
- Less memory usage

✗ Limitations:

- No AOP support
- No event handling
- No internationalization
- No integration with Spring Web MVC

```
Resource resource = new ClassPathResource("beans.xml");
```

```
BeanFactory factory = new XmlBeanFactory(resource);
```

```
MyBean obj = (MyBean) factory.getBean("myBean");
```

ApplicationContext

◆ Definition:

- **Advanced IoC container**, defined in `org.springframework.context.ApplicationContext`
- **Eagerly loads** all beans at startup
- **Superset** of `BeanFactory`

🧠 When to Use:

- For all **enterprise or web applications**
- Needed if using **AOP, Events, I18n, Spring MVC, Web support**

✅ Advantages:

- Event publishing
- AOP support
- Internationalization (i18n)
- Web integration
- Environment abstraction

Dependency Injection (DI)

What is Dependency Injection?

Dependency Injection (DI) is a design pattern used to implement **Inversion of Control (IoC)**, in which the control of creating and injecting dependent objects is given to the Spring container.

Real-Life Analogy

Imagine you're buying a laptop.

Instead of **manufacturing every part yourself**, you **buy it with all dependencies** (RAM, processor, OS) already assembled.

Spring works similarly:

When you request a bean, Spring injects all required beans (dependencies) into it.

Why DI is Important?

Without DI

Tightly coupled code

Difficult to test

Hard to switch implementations

More boilerplate code

With DI

Loose coupling

Easy to mock dependencies

Easy to switch with config

Clean, maintainable code

Types of Dependency Injection in Spring

Type	Description
1 Constructor Injection	Dependencies are provided via constructor
2 Setter Injection	Dependencies are set using setter methods
3 Field Injection	Dependencies are injected directly into fields

Constructor Injection (Recommended)

✓ Description:

- Dependencies are passed as **arguments to the constructor**.
- Makes the class **immutable** and **fully initialized** at the time of creation.

■ Example:

```
java
CopyEdit
@Component
public class Car {
    private Engine engine;

    @Autowired
    public Car(Engine engine) { // Constructor Injection
        this.engine = engine;
    }
}
```

Engine is another bean:

```
@Component
public class Engine {
    public Engine() {
        System.out.println("Engine created");
    }
}
```

🧠 Use When:

- Dependency is **mandatory**

- Class should be **immutable**
- Preferred for **unit testing** (mock via constructor)

Setter Injection

✅ Description:

- Dependencies are injected using **public setter methods**.
- More flexible; you can change or set dependencies **later**.

📘 Example:

```
@Component
public class Car {
    private Engine engine;

    @Autowired
    public void setEngine(Engine engine) { // Setter Injection
        this.engine = engine;
    }
}
```

🧠 Use When:

- Dependency is **optional**
- You need to **change dependencies** at runtime
- You want **flexibility** over strictness

Field Injection

✅ Description:

- Dependency is injected **directly into the field** using `@Autowired`.

📘 Example:

```
@Component
public class Car {
    @Autowired
    private Engine engine; // Field Injection
}
```

🧠 Use When:

- Quick development

- You want **less boilerplate**
- Not recommended for large or production systems because:
 - It makes unit testing harder
 - Breaks encapsulation
 - Cannot create immutable objects

When to Use What? – Interview Table

Type	When to Use	Pros	Cons
Constructor Injection	Required dependencies, better testability	Immutable objects, ensures complete initialization	More verbose
Setter Injection	Optional dependencies	Flexible, easy to change	Allows incomplete initialization
Field Injection	Quick, small components	Less code	Breaks encapsulation, hard to test

Spring Bean Configuration – In XML, Java, and Annotation-Based

Why Configuration Matters in Spring?

Spring needs to **know how to create and wire beans**. This is done through **Configuration Metadata**, which tells the Spring container:

- What beans to create
- What their dependencies are
- How to inject those dependencies

There are **three primary ways to configure Spring beans**:

XML-Based Configuration (Legacy Style)

Definition:

You write a configuration file (usually beans.xml) to declare all your beans and their dependencies using `<bean>` and `<property>` tags.

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd">

  <!-- Bean Definitions -->
```

```
<bean id="engine" class="com.example.Engine"/>
```

```
<bean id="car" class="com.example.Car">  
  <property name="engine" ref="engine"/>  
</bean>
```

```
</beans>
```

```
public class Car {  
    private Engine engine;  
  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }
```

```
    public void drive() {  
        engine.start();  
        System.out.println("Driving car...");  
    }  
}
```

```
public class Engine {  
    public void start() {  
        System.out.println("Engine started");  
    }  
}
```

When to Use XML Configuration?

✅ Use when:

- You want **separate config from code**
- You're working in **legacy or migration projects**
- You want centralized XML management

❌ Avoid when:

- Project is annotation-heavy

- You want modern, cleaner approaches

Java-Based Configuration (@Configuration class)

Definition:

Spring allows us to use **Java classes to configure beans** using `@Configuration` and `@Bean` annotations.

`@Configuration`

```
public class AppConfig {
```

```
    @Bean
```

```
    public Engine engine() {
```

```
        return new Engine();
```

```
    }
```

```
    @Bean
```

```
    public Car car() {
```

```
        return new Car(engine());
```

```
    }
```

```
}
```

```
public class MainApp {
```

```
    public static void main(String[] args) {
```

```
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        Car car = context.getBean(Car.class);
```

```
        car.drive();
```

```
    }
```

```
}
```

When to Use Java Config?

 Use when:

- You want **type-safety** and **IDE support**
- Need to use logic in bean creation
- Modern, modular configuration

✗ Avoid if:

- You want externalized or decoupled configuration

Annotation-Based Configuration (Component Scanning)

Definition:

Beans are defined directly in Java classes using annotations like:

- `@Component` – Marks a class as a bean
- `@Autowired` – Injects dependencies
- `@Service`, `@Repository`, `@Controller` – Specialized component stereotypes

`@Component`

```
public class Car {
```

```
    @Autowired
```

```
    private Engine engine;
```

```
    public void drive() {
```

```
        engine.start();
```

```
        System.out.println("Car is moving...");
```

```
    }
```

```
}
```

`@Component`

```
public class Engine {
```

```
    public void start() {
```

```
        System.out.println("Engine ignited!");
```

```
    }
```

```
}
```

`@ComponentScan("com.example")`

`@Configuration`

```
public class AppConfig {
```

```
    public static void main(String[] args) {
```

```
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);
```

```
        Car car = context.getBean(Car.class);
```

```
        car.drive();
```

```
}  
}
```

When to Use Annotation Config?

✅ Use when:

- You want **auto-detection of components**
- You prefer **less boilerplate** and **clean code**
- Most common in **modern Spring Boot applications**

❌ Avoid when:

- You want **clear externalized control**
- You need **manual bean wiring**

Comparison Table for Interview

Feature	XML Config	Java Config	Annotation Config
Format	External XML file	Java class with <code>@Configuration</code>	Java class with <code>@Component</code> , <code>@Service</code> , etc.
Type-Safety	❌ No	✅ Yes	✅ Yes
IDE Support	❌ Weak	✅ Strong	✅ Strong
Cleanliness	❌ Verbose	✅ Clean	✅ Cleanest
Dependency Injection	<code><property></code> , <code><constructor-arg></code>	Method return type	<code>@Autowired</code> , <code>@Inject</code>
Flexibility	✅ High	✅ Moderate	❌ Low
Use Case	Legacy systems, external config	Modular, testable configs	Auto-scanning modern apps

Spring AOP (Aspect-Oriented Programming)

What is AOP (Aspect-Oriented Programming)?

Aspect-Oriented Programming (AOP) is a programming paradigm that allows you to **separate cross-cutting concerns** from the main business logic.

🔍 **Cross-cutting concerns** are parts of a program that affect multiple layers of an application — such as:

- Logging

- Security
- Transactions
- Caching
- Performance monitoring
- Exception handling

In traditional OOP, these concerns get **scattered across many classes**, making the code:

- **Hard to maintain**
- **Hard to test**
- **Tightly coupled**

AOP solves this problem by allowing you to write these concerns **once in a reusable way** and apply them where needed.

Why AOP is Important in Spring?

Spring AOP helps in:

Benefit	Explanation
✓ Loose Coupling	Business logic is separated from auxiliary concerns
✓ Code Reusability	Write logging or security once, reuse across modules
✓ Better Maintainability	All concerns are centralized
✓ Declarative Approach	Use annotations to apply behavior without modifying core logic

ACID Properties of Transactions

ACID is an acronym that describes the **key properties of a reliable database transaction**. These ensure the **integrity, consistency, and stability** of the data when multiple operations are performed — especially in critical systems like **banking, e-commerce, and finance**.

Letter	Full Form	Meaning
A	Atomicity	All operations in a transaction are treated as a single unit
C	Consistency	Database must remain in a valid state before and after transaction
I	Isolation	Concurrent transactions must not interfere with each other
D	Durability	Once a transaction is committed, the data remains permanent

Atomicity – All or Nothing

- ✓ **Definition:**

Atomicity ensures that **either all operations within a transaction are completed successfully, or none are applied.**

 Example:

In a money transfer:

- Debit ₹10,000 from Account A
- Credit ₹10,000 to Account B

If the system crashes after debiting but before crediting — **Atomicity rolls back** the debit. No partial change is allowed.

 In Spring:

If an exception occurs in a @Transactional method, Spring rolls back all previous operations in that transaction.

2 Consistency – Valid State In, Valid State Out

 Definition:

A transaction must **take the database from one valid state to another**. It ensures that **data integrity rules and constraints** are not violated.

 Example:

Suppose a table has:

- balance ≥ 0 constraint

If a transaction reduces balance to -500, it **violates consistency**.

Transaction will **rollback automatically** or be rejected by the database.

 In Spring:

Using validations, exception handling, and constraints in entities ensures consistency.

3 Isolation – Don't Disturb Me

 Definition:

Isolation ensures that **concurrent transactions don't interfere** with each other. Each transaction should behave as if it is the **only transaction** running in the system.

 Example:

Two users try to book the **last movie ticket** at the same time.

If isolation isn't ensured:

- Both users may succeed 

- Leads to **double booking** ❌

🔧 How Achieved?

Using **isolation levels** like:

- READ_COMMITTED
- SERIALIZABLE
- READ_UNCOMMITTED
- REPEATABLE_READ

These define how and when changes made by one transaction become visible to others.

🔑 Durability – Permanent Changes

✅ Definition:

Once a transaction is **committed**, its changes are **permanent**, even if the system crashes immediately afterward.

📦 Example:

If a user buys a product and transaction is committed:

- Even if the server restarts
- That purchase record **must remain saved**

Durability is usually ensured by the **database engine** writing to **disk (log files, journaling, etc.)**.

🔧 In Spring:

The underlying database (like MySQL, PostgreSQL) handles this. Spring ensures a transaction is properly committed.

What is Transaction Management?

Transaction Management is the process of **coordinating and managing a sequence of operations** (usually database operations) so that they behave as **one single unit of work** — a **transaction**.

If **any one step fails**, the **entire transaction is rolled back** — meaning no change is made to the database.

✅ Real-World Analogy

Imagine you're transferring ₹10,000 from Account A to Account B.

Steps involved:

1. Debit ₹10,000 from Account A ✅
2. Credit ₹10,000 to Account B ✅

If step 1 succeeds, but step 2 fails (e.g., network error), your money vanishes. ❌

With transaction management, either both succeed, or none is applied.

Purpose of Transaction Management

Purpose	Description
✓ Ensure Data Consistency	Keeps data valid even if failures occur
✓ Atomicity	All operations within a transaction are either completed or none at all
✓ Integrity	Prevents partial updates to the database
✓ Error Handling	Automatically rolls back changes on failure
✓ Resource Management	Locks, concurrency, and rollback are properly managed

Why Do We Need Transaction Management?

In real applications:

- Multiple database calls happen for a single business operation.
- Failures like power cuts, exceptions, or system crashes may occur.
- Without transactions, this leads to:
 - Corrupted data
 - Inconsistent state
 - Revenue loss